Tech Challenge — Fase 1

Sistema Integrado de Atendimento e Execução de Serviços — Oficina Mecânica (MVP do back-end)

1. Identificação da entrega

Curso	SOAT — Software Architecture
Fase	1
Número do grupo	95

Integrantes

Nome completo	RM	Username no Discord
Gabriel Mauricio Zanette Gaidzinksi	RM378808	zgthemaverick

Linguagem e principais tecnologias

- **Linguagem:** C# (.NET 10)
- **API:** ASP.NET Core Web API (REST), autenticação JWT Bearer
- **Documentação da API:** OpenAPI + Swagger UI
- **Persistência:** Entity Framework Core 10 + PostgreSQL 17 (Npgsql)
- **Testes:** xUnit, EF Core InMemory, WebApplicationFactory, Coverlet + ReportGenerator
- **Infraestrutura:** Docker e Docker Compose
- **Qualidade e segurança:** SonarQube, Trivy, `dotnet list package --vulnerable`
- **Arquitetura:** monólito em camadas com DDD; diagramas C4 no Structurizr

2. Repositório do projeto

Repositório (público)	https://github.com/ZgMaverick/ProjetoFiap_Oficina
Branch da entrega	`dev` (o mesmo código foi integrado na `main` pelo PR #1)
Commit da entrega	80fd99731d320751d3dec8d5cae27fc01d1cde0b

A avaliação deve considerar exclusivamente a versão acima. A branch `main` é protegida e só recebe alterações por Pull Request.

Onde encontrar cada item no repositório

Requisito	Onde está
README completo	README.md
Instruções de execução local	README, seções "Pré-requisitos" e "Como executar" (`cp .env.example .env` e `docker compose up --build`)
Testes automatizados (mín. 80%)	`tests/` e coverage-report/: branches 81,8%, linhas 93,9%, métodos 96,4% (domínio e aplicação)
Documentação da API	Seção "Documentação da API" abaixo
Análise de vulnerabilidades	Seção "Análise de vulnerabilidades" abaixo
Credenciais	Nenhuma no repositório: senhas e chave do JWT vêm do arquivo `.env` (fora do Git), a partir do modelo .env.example

Documentação da API (Swagger / OpenAPI)

- **Swagger UI:** com a aplicação rodando, em `http://localhost:8080/swagger`. O botão **Authorize** aceita o token JWT obtido em `POST /api/auth/login`.
- **Documento OpenAPI 3.1 (ao vivo):** `http://localhost:8080/openapi/v1.json`
- **Cópia exportada no repositório:** docs/api/openapi.json, que pode ser aberta em https://editor.swagger.io sem rodar o projeto.

Grupos de endpoints: `Auth`, `Clientes`, `Veiculos`, `Servicos`, `PecasInsumos`, `OrdensServico` (administrativos, exigem JWT) e `Acompanhamento` (público, usado pelo cliente).

Análise de vulnerabilidades

Relatórios em docs/seguranca/:

Análise	Ferramenta / relatório	Resultado
Pacotes NuGet (incluindo transitivos)	`dotnet list package --vulnerable --include-transitive` → pacotes-nuget.txt	Nenhum pacote vulnerável
Imagem Docker da API (HIGH/CRITICAL)	Trivy → imagem-docker.txt	0 vulnerabilidades (Ubuntu 24.04 e runtime .NET 10.0.14)
Análise estática do código	SonarQube → Vulnerabilidades.png	Painel com Quality Gate, bugs, vulnerabilidades e cobertura

3. Vídeo da entrega

YouTube (público):

PENDENTE: inserir o link do vídeo no YouTube

4. Documentação DDD

- **Miro:** https://miro.com/app/board/uXjVEXtrgrQ=/?share_link_id=578425858571
- **No repositório:** docs/ddd/

Conteúdo pedido	Onde está
Event Storming da criação da OS, do acompanhamento da OS e da gestão de peças e insumos	event-storming.md (seções 1 a 3), event-storming.png e a versão marcada com o que ficou fora do MVP, event-storming-mvp.png
Eventos de domínio, comandos, políticas e atores	event-storming.md, seção "Resumo"
Agregados, entidades, objetos de valor e bounded contexts	modelo-de-dominio.md
Diagramas	Domain Storytelling (domain-storytelling.png; marcado: domain-storytelling-mvp.png) e C4 (c4.png, workspace.dsl)
Linguagem ubíqua e glossário	linguagem-ubiqua.md e glossario.pdf
Relação entre a documentação e o código	modelo-de-dominio.md, seções "Relação entre documentação e código" e "Decisões do MVP"